

Codex 个人招聘工作台 | 整体架构与技术方案 V1.0

一、方案结论

第一版采用：

单一 Codex 主 Agent + 任务型 Skills + Markdown 主档案 + Python 文件处理脚本 + 自动生成索引

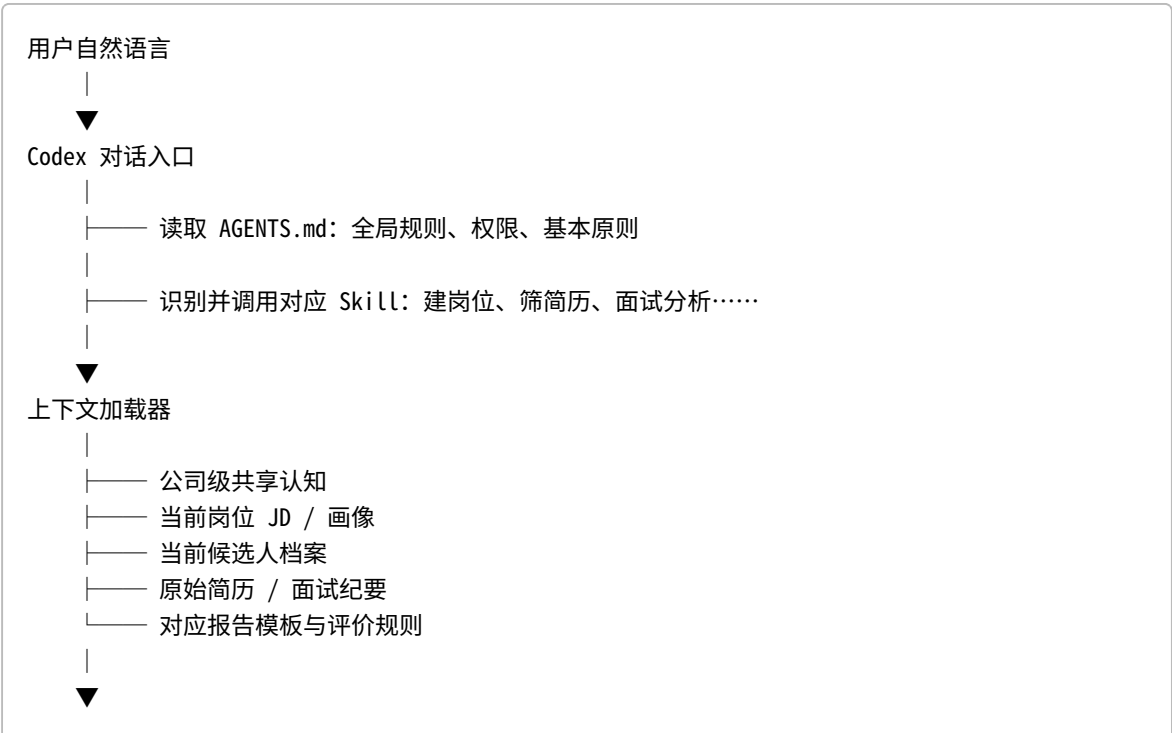
不做：

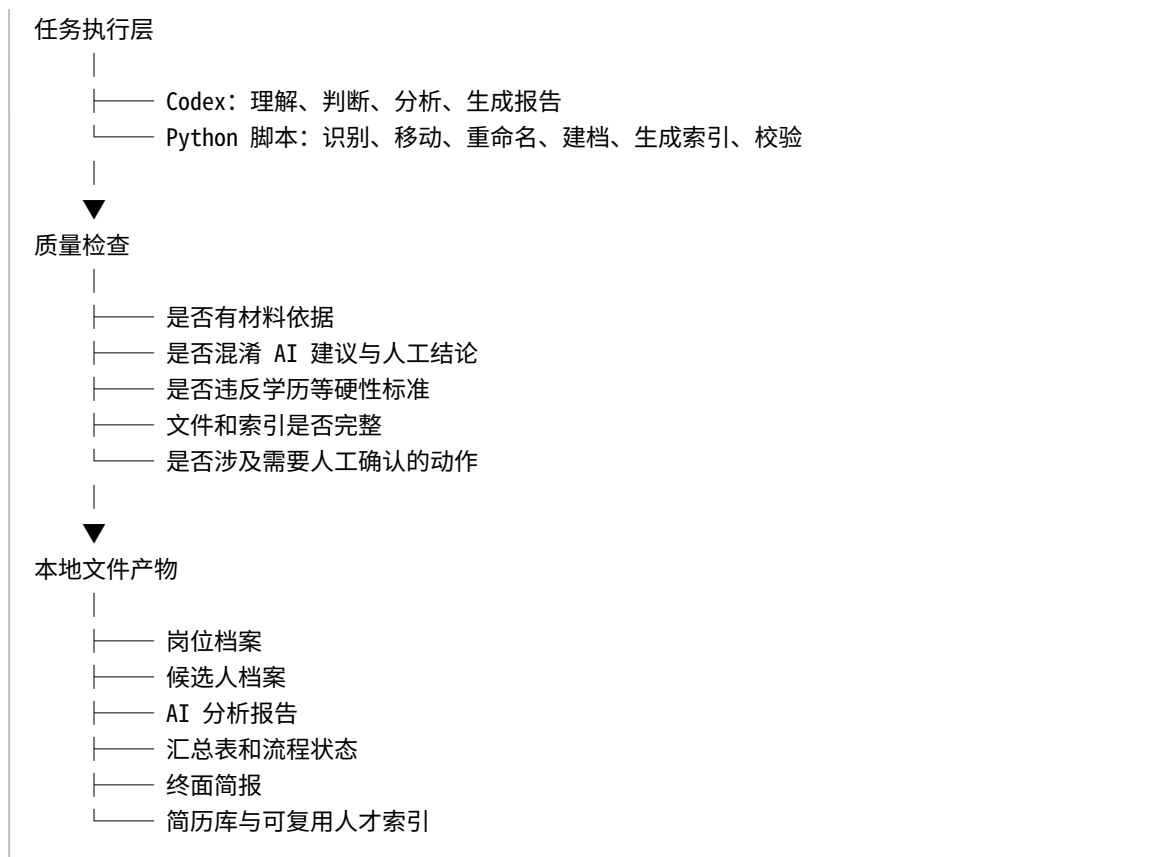
- Web 前端；
- 后端服务；
- 数据库；
- 自建模型 API；
- 多 Agent 编排；
- 后台定时任务；
- ATS 集成。

用户打开招聘项目文件夹后，直接在 Codex 中用自然语言操作。

Codex 本身可以读取和修改当前项目目录、运行本地命令； `AGENTS.md` 可以作为项目级长期规则，项目内的 `.agents/skills` 可以承载可重复的具体工作流。Skills既可以由自然语言隐式触发，也可以通过 `$技能名` 显式调用。

二、整体架构





整个系统分成六层：

1. **交互层**：Codex自然语言对话；
2. **规则层**：AGENTS.md；
3. **工作流层**：Skills；
4. **知识与状态层**：本地 Markdown、PDF、TXT、CSV；
5. **确定性执行层**：Python脚本；
6. **校验与审计层**：待确认事项、变更记录、自动检查。

三、为什么不只写一个超级 Prompt

一个大 Prompt 会产生几个问题：

- 每次任务都塞入大量无关规则；
- 简历筛选和面试分析互相干扰；
- 后续修改某个环节容易影响其他环节；
- 上下文越来越长，Codex使用成本和不稳定性都会增加。

AGENTS.md 适合放简短、长期、所有任务都必须遵守的规则。Codex启动任务前会读取它，但项目指令默认有上下文大小限制，因此不适合把所有详细提示词都塞进去。具体任务应拆成Skills，并在匹配任务时才加载详细说明。

因此采用以下分工：

载体	放什么
AGENTS.md	系统宪法、权限边界、事实来源、禁止事项
SKILL.md	某一项工作的执行步骤、输入输出和触发条件
references/	详细评价标准和长提示词
assets/	报告模板、Markdown模板
scripts/	文件移动、重命名、索引和校验代码
业务文件夹	岗位、候选人、面试和招聘结果

你现有的终面简报 Prompt 不放进根目录 AGENTS.md，而应作为“生成终面简报”Skill的详细参考规则。

四、文件夹总体架构

```

Codex招聘工作台/
|
|—— AGENTS.md
|—— README.md
|—— .gitignore
|
|—— .agents/
|   |—— skills/
|       |—— create-position/
|       |—— process-resumes/
|       |—— confirm-screening/
|       |—— analyze-interview/
|       |—— generate-final-brief/
|       |—— calibrate-position/
|       |—— close-candidate/
|       |—— search-history/
|
|—— 00_公司认知/
|   |—— 公司与业务.md
|   |—— 通用招聘标准.md
|   |—— 能力特质定义.md
|   |—— 已确认长期认知.md
|
|—— 01_待处理/
|   |—— 简历/
|   |—— 面试纪要/
|   |—— 待确认/
|
|—— 02_岗位/
|   |—— C端产品经理/
|   |—— Android开发工程师/
|   |—— AI Agent Harness产品经理/

```

```

|
|—— 03_简历库/
|   |—— C端产品经理/
|   |—— Android开发工程师/
|   |—— AI Agent Harness产品经理/
|
|—— 04_全局索引/
|   |—— 岗位清单.csv
|   |—— 全部候选人.csv
|   |—— 可复用候选人.csv
|   |—— 待确认事项.md
|
|—— 05_共享模板/
|   |—— 岗位模板.md
|   |—— 候选人总览模板.md
|   |—— 简历分析模板.md
|   |—— 面试报告模板.md
|   |—— 候选人汇总表模板.md
|   |—— 终面简报模板.md
|
|—— 06_系统/
|   |—— recruiter/
|   |   |—— ingest.py
|   |   |—— files.py
|   |   |—— indexes.py
|   |   |—— archive.py
|   |   |—— validators.py
|   |   |—— cli.py
|   |—— schemas/
|   |—— tests/
|   |—— cache/
|   |—— config.yaml
|
|—— 07_运行记录/
|   |—— actions.jsonl
|   |—— errors.jsonl
|   |—— 系统变更记录.md

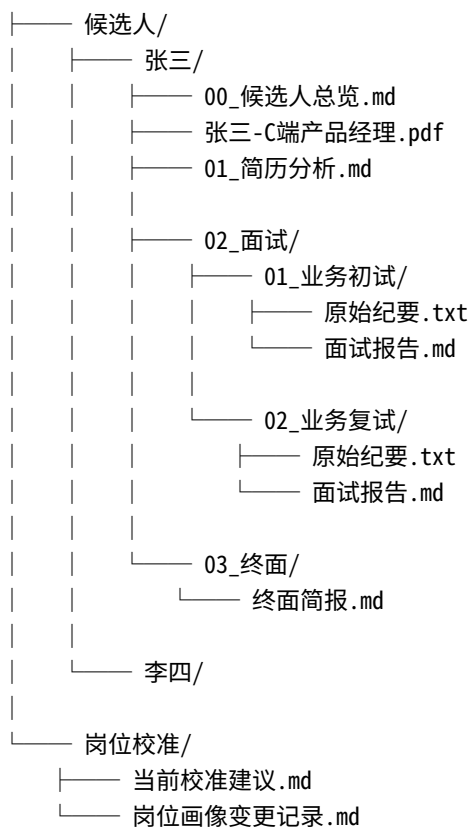
```

五、单个岗位文件夹结构

```

02_岗位/
|—— C端产品经理/
|   |—— 岗位.md
|   |—— 候选人总表.md
|   |—— 候选人总表.csv
|   |—— 已结束候选人索引.md
|

```



这里的原则是：

- 当前岗位只放“推进中”和“待定”的完整候选人档案；
- 流程结束后，候选人完整文件夹移动到 `03_简历库/对应岗位/`；
- 原岗位只在 `已结束候选人索引.md` 中保留简短记录；
- 初筛淘汰者也进入简历库，但只保留简历和简短结论；
- 所有简历统一命名为 `姓名-岗位.扩展名`。

六、核心文件设计

1. 岗位.md

岗位文件既是人看的文档，也是Codex后续任务的正式上下文。

建议使用 Markdown 正文加 YAML 头部：

```
---
position_id: POS-2026-001
position_name: C端产品经理
status: 待校准
created_at: 2026-07-16
updated_at: 2026-07-16
image_version: 1
```

C端产品经理

一、JD

岗位职责

.....

任职要求

.....

二、岗位画像

这个岗位负责什么

.....

要招什么样的人

.....

核心能力

.....

关键经历

.....

简历筛选重点

.....

面试重点判断

.....

可放宽条件

.....

三、公司通用标准

本岗位继承《00_公司认知/通用招聘标准.md》。

- 第一学历原则上要求985本科；
- 211本科仅特别优秀时建议破格，并必须说明理由；
- 非985、211或未写本科院校，AI一律建议淘汰；
- 公司通用能力特质主要在面试阶段验证。

四、待校准事项

-

五、确认与变更记录

- 2026-07-16：创建待校准版本。

岗位状态可以使用：

- 待校准；
- 招聘中；
- 暂停；
- 已结束。

2. 00_候选人总览.md

总览是候选人的当前状态入口，但不覆盖各轮原始记录。

```
---
candidate_id: CAN-2026-0001
name: 张三
position: C端产品经理
source: Boss直聘
current_stage: 待业务筛选
ai_recommendation: 推
human_decision: 待确认
reusable: false
created_at: 2026-07-16
updated_at: 2026-07-16
---

# 张三 | 候选人总览

## 一、当前状态

- 当前轮次：待业务筛选
- AI建议：推
- 人工结论：待确认
- 当前流程结论：推进中

## 二、基本信息

.....

## 三、简历阶段判断

.....

## 四、历轮面试结论

.....

## 五、当前主要优势

.....
```

六、当前风险与未验证项

.....

七、重要变更记录

.....

必须同时保留：

- `ai_recommendation`：AI建议；
- `human_decision`：人工正式结论。

AI不能通过重写报告，偷偷把人工结论覆盖掉。

3. 候选人总表

同时维护两个版本：

- `候选人总表.md`：方便你直接阅读；
- `候选人总表.csv`：方便排序、筛选和自动生成其他报告。

主要字段包括：

排序	姓名	AI建议	人工结论	当前轮次	核心匹配点	主要风险	来源
----	----	------	------	------	-------	------	----

不打具体分数，四档统一排序。

这两张表不手动维护，而是由脚本读取每个候选人总览中的 YAML 信息后重新生成，避免表格和候选人档案不一致。

七、Skills workflow 架构

每项核心任务做成一个独立 Skill。官方建议Skill聚焦一个工作，并明确输入、输出和执行步骤；Skill还可以携带脚本、参考资料和模板。

1. `create-position`

触发语义：

- 新建岗位；
- 梳理岗位画像；
- 和我讨论这个岗位；
- 根据这个JD建岗。

工作方式：

1. 读取公司共享认知；
2. 读取用户提供的JD、描述或材料；
3. 进入岗位讨论阶段；
4. 主动整理、追问和适度质疑；
5. 区分已确认信息、暂时假设和待验证事项；
6. 未得到用户确认前，不写正式岗位文档；
7. 用户确认后创建岗位目录和 岗位.md；
8. 扫描一次历史可复用候选人。

这个Skill的“讨论阶段”就是业务层面的计划模式。

2. process-resumes

触发语义：

- 处理今天下载的简历；
- 筛一下新简历；
- 把待处理简历整理一下。

工作流程：

1. 扫描 01_待处理/简历；
2. 提取简历文本；
3. 识别姓名和目标岗位；
4. 无法判断的移动到 待确认；
5. 统一重命名；
6. 读取对应岗位资料；
7. 检查公司通用学历规则；
8. 生成固定格式简历分析；
9. 给出强推、推、建议待定、建议淘汰；
10. 更新统一排序；
11. 生成本批次汇总；
12. 等待用户批量确认人工结论。

Codex负责候选人判断，Python脚本负责识别、移动、重命名、创建目录和更新索引。

3. confirm-screening

负责处理：

“前3个推进，第4个待定，其他淘汰。”

它将：

- 写入人工结论；
- 推进和待定者建立完整档案；

- 淘汰者移入简历库；
 - 更新岗位候选人总表；
 - 保留AI建议与人工结论之间的差异。
-

4. analyze-interview

触发语义：

- 分析新面试纪要；
- 帮我整理张三的一面；
- 处理待处理区里的面试转写。

工作流程：

1. 扫描 01_待处理/面试纪要；
 2. 自动识别候选人、岗位和轮次；
 3. 找到对应候选人档案；
 4. 保存原始纪要；
 5. 生成本轮面试报告；
 6. 区分面试官评价、Codex独立分析、人工正式结论；
 7. 更新候选人总览；
 8. 更新流程状态；
 9. 无法匹配时放入待确认。
-

5. generate-final-brief

读取：

- 岗位文档；
- 原始简历；
- 简历筛选分析；
- 每轮原始面试纪要；
- 每轮面试报告；
- HR补充信息；
- 当前候选人总览。

然后根据现有终面简报Prompt生成最终材料。该Prompt中已经定义了岗位标准优先、按候选人年限调整、证据优先、区分事实和判断、处理材料冲突等完整要求，可以直接作为首版规则基础。

6. calibrate-position

触发语义：

- 看看这个岗位画像有没有问题；
- 根据最近的面试结果校准画像；
- 为什么推荐的人业务一直不要？

工作流程：

1. 读取岗位画像历史版本；
2. 对比AI筛选建议和人工结论；
3. 对比面试结果与最初判断；
4. 汇总重复出现的通过、淘汰和分歧原因；
5. 判断问题更可能来自画像、筛选、面试标准还是个别案例；
6. 生成校准建议；
7. 展示建议修改前后的差异；
8. 用户确认后再更新 `岗位.md`。

7. `close-candidate`

当用户说：

- 张三流程结束；
- 张三拒绝Offer；
- 李四已入职；
- 这个候选人淘汰归档。

Skill完成：

- 写入最终轮次和结束原因；
- 自动判断是否属于可复用候选人；
- 将完整档案移动至简历库；
- 在原岗位留下简短索引；
- 更新全局人才索引。

8. `search-history`

支持：

- 找以前不错的产品人选；
- 找通过业务面试但没入职的人；
- 新岗位历史人选扫描；
- 查某个人以前聊到哪一轮。

它只读取历史资料并给出推荐，不自动将候选人加入新岗位流程。

八、AGENTS.md 应该包含什么

根目录 `AGENTS.md` 只放所有任务都必须遵守的规则。

建议包括：

1. 系统定位

- 本项目是个人招聘主档案；
- 本地文件是唯一事实来源；
- Codex对话不是正式存储；
- 每次重要结论必须落文件。

2. 工作原则

- 先读取现有资料，再执行；
- 不猜测缺失信息；
- 判断必须有材料依据；
- 原始简历和面试纪要不得被AI报告替代；
- AI建议和人工正式结论必须分开；
- 不使用具体评分；
- 简历统一使用四档建议。

3. 权限规则

可以直接执行：

- 整理；
- 分析；
- 生成报告；
- 建立索引；
- 将正常处理完成的文件从待处理区移入确定位置。

必须确认：

- 删除原始资料；
- 修改岗位画像；
- 修改人工正式结论；
- 跨岗位移动；
- 修改公司级共享认知；
- 根据新材料改变已确认的候选人结论。

4. 学历规则

直接写入已确认的公司统一标准。

5. 文件规则

只写少量核心规则，具体细节由脚本负责，不把所有文件边角规则塞入AGENTS.md。

九、技术方案

1. 运行环境

推荐使用：

- Codex桌面端打开项目根目录；
- Codex CLI或IDE扩展作为备用；
- 使用ChatGPT账号登录，不需要自行配置模型API。

Codex桌面端适合本地文件和较长任务；CLI也可以直接读取、编辑本地项目和运行命令。Codex可直接使用ChatGPT账号登录，但实际使用仍受套餐用量和credits限制，所以“不配置API”不等于完全没有模型使用成本。

2. Python 技术栈

建议使用：

- Python 3.11+
- `uv` 管理虚拟环境和依赖
- `PyMuPDF`：提取PDF文字
- `python-docx`：读取Word
- `PyYAML`：处理Markdown头部和配置
- `rapidfuzz`：姓名、岗位名称模糊匹配
- `charset-normalizer`：文本编码识别
- `pytest`：自动化测试

不使用：

- Django；
- FastAPI；
- React；
- PostgreSQL；
- Redis；
- 向量数据库。

第一版没有服务端需求，这些东西只会增加维护成本。

3. 脚本与AI的职责分工

脚本负责确定性动作

- 文件扫描；
- PDF和Word文本提取；
- 文件哈希；
- 文件重命名；
- 创建目录；

- 文件移动；
- Markdown frontmatter读写；
- CSV索引生成；
- 路径检查；
- 重复文件识别；
- 工作区完整性校验。

Codex负责需要理解的动作

- 理解岗位；
- 质疑画像；
- 分析候选人；
- 判断匹配度；
- 给出四档建议；
- 生成业务摘要；
- 分析面试；
- 识别评价矛盾；
- 生成终面简报；
- 提出岗位校准建议。

原则是：

能用明确代码稳定完成的动作，不依赖模型自由发挥；需要业务判断的动作，才交给Codex。

4. 内部命令

用户不需要记命令，但Codex可以在后台调用：

```
python -m recruiter ingest-resumes
python -m recruiter ingest-interviews
python -m recruiter rebuild-index
python -m recruiter archive-candidate
python -m recruiter validate
```

例如用户说：

“处理新简历。”

Codex先调用 `ingest-resumes` 完成扫描和文本准备，再读取结果进行AI分析，最后调用 `rebuild-index` 更新岗位汇总。

5. 状态存储

不引入数据库。

使用：

- Markdown YAML头部保存单个岗位和候选人的正式状态；
- Markdown正文保存人类可读信息；
- CSV保存自动生成的全局索引；
- JSONL保存运行和变更记录。

其中：

- Markdown是主档案；
- CSV是可重建索引；
- JSONL是审计记录；
- 缓存可以随时删除并重新生成。

十、权限与确认机制

需要确认的动作，不应只在对话中问一句“是否确认”，而应形成结构化待确认事项。

例如：

待确认事项

2026-07-16 | 修改候选人结论

候选人：张三
岗位：C端产品经理

当前人工结论：推
新AI建议：建议待定

变化原因：
- 新版简历删除了一段核心项目经历；
- 当前材料不足以支持独立负责判断。

待执行动作：
- 修改AI建议；
- 更新岗位排序；
- 保持原人工结论不变。

状态：待确认

用户说“确认”后执行，并写入变更记录。

这样即使对话中断，也不会丢失尚未处理的关键动作。

十一、质量校验机制

第一版至少实现以下校验：

文件校验

- 岗位目录是否有 `岗位.md` ；
- 推进中候选人是否有总览和简历；
- 面试报告是否对应原始纪要；
- 简历库路径是否有效；
- 候选人索引是否存在失效链接。

业务校验

- 非985、211候选人是否错误地被建议推进；
- 211破格是否写明具体原因；
- AI是否将“未验证”写成“符合”；
- 是否大段复述简历；
- 是否给出无证据评价；
- 是否把面试官评价写成AI自己的结论；
- 是否覆盖了人工正式结论。

workflow 校验

- 需要确认的修改是否被直接执行；
- 流程结束候选人是否已归档；
- 岗位候选人总表与单人档案是否一致；
- 新增材料后相关索引是否重新生成。

十二、测试方案

不要用“生成出来看着不错”作为验收。

应准备一套脱敏测试材料：

- 2个真实岗位；
- 每个岗位10—20份历史简历；
- 若干通过和淘汰候选人；
- 至少5份面试转写；
- 2—3个终面案例；
- 已知的人工筛选结论和面试结果。

测试分三类：

1. 规则测试

验证学历、文件移动、状态更新、权限确认是否符合规则。

2. 输出测试

验证报告是否包含固定字段、是否有依据、是否出现废话和幻觉。

3. 效果测试

比较：

- AI建议与人工结论的差异；
- AI推荐后的业务面试通过情况；
- 面试结果能否正确回流到画像校准建议；
- 完成同一批工作所需人工操作是否减少。

模型生成结果不适合逐字对比，测试应看结构、关键结论和证据是否符合，而不是要求每次文字完全一致。

十三、数据安全

候选人简历和面试纪要包含大量个人信息，建议：

- 不把真实简历和转写提交到公开GitHub；
- Git只管理框架、Skills、模板和脚本；
- 候选人数据目录加入 `.gitignore`；
- 使用本地磁盘加密和可靠备份；
- 不在Skill或日志中重复写入无关身份证号、住址等敏感信息。

如果使用个人版ChatGPT或Codex，内容可能会用于模型改进，除非关闭相关训练设置；Codex还存在单独的完整环境数据控制。Business、Enterprise和API数据默认不用于训练。处理真实招聘数据前，应先检查并调整相关数据控制。

十四、开发顺序

阶段一：搭底座

先完成：

- 根目录结构；
- `AGENTS.md`；
- 公司通用标准；
- Markdown状态格式；
- 文件读取、移动、重命名和索引脚本；
- 权限和待确认机制。

阶段二：建岗位

完成：

- 岗位讨论Skill;
- 待校准和已确认状态;
- 岗位.md 生成;
- 历史候选人首次扫描。

阶段三：简历主链路

完成：

- 简历批量读取;
- 岗位匹配;
- 四档AI建议;
- 统一排序;
- 业务摘要;
- 汇总表批量确认;
- 淘汰归档。

这一阶段是第一版最重要的价值验证。

阶段四：面试主链路

完成：

- 面试纪要自动匹配;
- 动态识别面试轮次;
- 三层面试报告;
- 候选人总览更新;
- 状态推进。

阶段五：终面简报

将现有终面简报Prompt接入Skill，使用真实案例校准篇幅、判断尺度和输出结构。

阶段六：归档和人才复用

完成：

- 流程结束归档;
- 可复用人才自动标记;
- 新岗位历史候选人扫描;
- 自然语言历史查询。

阶段七：岗位校准

最后接上：

- 筛选判断；
- 人工结论；
- 面试结果；
- 最终录用结果；

形成岗位画像校准闭环。

十五、最终推荐

第一版不要追求“全自动招聘Agent”。

真正应交付的是：

一套稳定的个人招聘 Harness：Codex负责理解和判断，Skills负责流程，脚本负责确定性操作，本地文件负责保存事实，人负责关键确认。

最重要的技术决策有四个：

1. 不做网页和后端；
2. 不用数据库，Markdown是主档案；
3. 不做一个超级Prompt，而是拆成任务Skills；
4. 不让Codex自由完成所有文件操作，确定性动作交给脚本。

这样既保留自然语言的友好，又不会因为完全依赖对话和模型临场发挥，最后重新变成一堆不可控、不可复用的AI输出。